Parlianote

Making the parliamentary record searchable, citable and checkable by postcode

36118 Applied Natural Language Processing, UTS | Technical report | Group 11 (Team Phoenix)
Krishna Kumar Thirugnanavel, Sidharth Joly, Brian Shimmer, Malavika Rajesh, Prathamesh Nemade, Milad Kianfar
October 2026

Executive summary

Everything an MP says in the House of Representatives is on the public record, but 27 years of Hansard transcripts are too long and too dense for most voters to use, so MPs tend to be judged on headlines and advertising. **Parlianote** is a web application that lets a user type a postcode and see their MP, the issues that MP speaks about and how that mix has changed, a verbatim summary of what the MP said on any issue (every sentence carries a Hansard citation), an estimate of how directly ministers answer Question Time questions, and the pledges they made.

The system is built on 370,861 MP speeches (1998 to July 2025) and combines an LDA topic model (coherence 0.459), BM25 retrieval re-ranked by a cross-encoder, extractive summarisation, a directness classifier trained on 120 hand-labelled question-answer pairs, and a rule-based pledge detector. Topic discovery and retrieval work well. The directness estimate is much weaker than the poster we presented earlier implied: among opposition questions it separates evasive from non-evasive answers with about 68% accuracy against a 58% majority baseline on 64 pairs, so the app reports it as a screening signal with confidence intervals, never as a verdict on an individual. We also found and fixed a data-validity problem that had silently dropped 69 recently elected MPs. Section 8 lists what we would do next.

1. Project objectives and scope

1.1 Problem and users

Hansard is public, but it is decades of dense, unsearchable text. Most voters therefore judge their MP on headlines or advertising rather than on the record, and there is no easy way to track which MPs are raising the issues a voter cares about. Existing tools count votes or find keyword mentions; neither shows what an MP actually said, in context, with a way to check it. Our primary users are citizens who want to understand their own representative. Advocacy groups and journalists are secondary users who want to find and cite what MPs have said on an issue.

1.2 Objectives

- **O1.** Resolve a postcode to the current MP for that electorate.
- **O2.** Show what an MP talks about and how their focus has shifted over time.
- **O3.** Answer "what has my MP said about X?" with verbatim passages and a citation for every sentence shown.
- **O4.** Estimate how directly answers address the question asked at Question Time, and surface the pledges ministers make.
- **O5.** Evaluate every component against labelled data, report results honestly, and state limits in the interface.
- **O6.** Deliver the above as a usable web application.

1.3 Scope and relationship to earlier work

In scope: House of Representatives only; 1998 to 31 July 2025 (Question Time analysis 2011 to 2025, see Section 3.4); English text; current-electorate postcode lookup; extractive (not generated) summaries. **Out of scope:** the

Senate, voting records, committee evidence, live updates, a user study with members of the public, and public deployment.

This report builds on the Idea Napkin proposal and the Part A poster (group mark 84/100). The poster pitched a "dodge index" and listed evaluation methods such as inter-annotator agreement and a cross-party bias audit. In this report we state which of those were actually carried out, and where the evidence is weaker than the poster suggested we say so.

2. Project phases and contributions

Phase	What was done	Main outputs
1. Ideation	Problem framing, dataset choice, Idea Napkin on Miro, instructor feedback	Napkin; project scope
2. Exploration and phase-1 analysis	Cleaning, postcode join, LDA, zero-shot issue classification, RAG retrieval pilot for one MP	Notebook (parlianote_analysis.ipynb); insights document
3. Poster (AT2 Part A)	Communicating goals, techniques and findings	Group poster (84/100)
4. Question-answer analysis	Pairing questions with answers, labelling, directness models, pledge detection	src/ pipeline; labelled sets; results tables
5. Web application	Hybrid retrieval, topic views, directness and pledge views, citation design	app/app.py (Streamlit)
6. Evaluation and report	Cross-validation, retrieval comparison, data-quality audit, this report	results/ folder; this document

Table 1. Project phases.

Contributions by team member. Each member's contribution is summarised in Table 2; the team's self and peer ratings are submitted separately on SparkPlus.

Team member	Contribution
Krishna Kumar Thirugnanavel	[to be completed by the team]
Sidharth Joly	[to be completed by the team]
Brian Shimmer	[to be completed by the team]
Malavika Rajesh	[to be completed by the team]
Prathamesh Nemade	[to be completed by the team]
Milad Kianfar	[to be completed by the team]

Table 2. Individual contributions (to be completed by the team before submission).

Declaration of AI assistance. We used an AI coding assistant (Claude Code, Anthropic) to help write and debug the code in this project, and the same assistant produced the single-annotator labels for the 120 question-answer pairs and 90 sentences described in Section 4. The team is responsible for the content of this report. Because those labels were not produced by independent human annotators, Section 5 treats the evaluation as preliminary.

3. Data sources

3.1 Hansard corpus

Katz and Alexander (2025) parsed every sitting day of the House of Representatives from 2 March 1998 to 31 July 2025 from the official XML into a single Parquet file (579.6 MB, 647,852 rows). Each row is one utterance with the speaker, electorate, party, date, page number, speech number and body text, plus flags for questions, answers, interjections and member validity. We keep rows that have an electorate, are not interjections, are not flagged as a non-member, and contain more than 30 characters, giving **370,861 speeches** from 1998 to 2025.

3.2 Postcode lookup

The Hansard data records electorate names but not postcodes. We joined it to the open Australian Postcodes dataset (Proctor, n.d.), which maps 3,165 postcodes to federal electorates. The current MP for an electorate is taken to be its most recent speaker in the corpus.

3.3 Annotated data created by the team

- **Directness labels:** 120 Question Time pairs sampled at random, about 10 per year from 2011 to 2025, labelled Direct (52), Partial (39) or Evasive (29) from the visible question and the first 110 words of the answer.
- **Pledge labels:** 90 sentences from ministerial answers (45 flagged by our rules, 45 not), labelled Specific commitment (12), Vague or hedged intention (25) or Not a commitment (53).
- **Topic and issue checks:** 40 random speeches graded for correct issue assignment (phase 1) and a set of retrieval queries judged for relevance (Section 5.4).

3.4 Data quality issues found and how we handled them

Issue	Evidence	Handling
Answer flags unreliable before 2011	In 2010 only 287 answers are flagged against 1,208 questions; from 2011 the counts match (1,276 vs 1,496 in 2011)	Question-answer analysis restricted to 2011-2025 (16,256 pairs); earlier years excluded
Member flag blank for MPs elected from 2022	8,658 genuine speeches from 69 MPs (2022-2025) were removed by our first filter; the Brisbane postcode returned the previous MP	Filter changed to exclude only member = 0; corpus grew from 362,203 to 370,861 speeches
Electorate redistributions	Only 143 of 166 Hansard electorates (86%) appear in the current postcode data (e.g. Batman is now Cooper)	Old seats stay searchable by MP name, not by postcode
Postcodes spanning electorates	A postcode can map to several electorates	App asks the user to choose and links to the AEC for confirmation
Corpus ends 31 July 2025	Newest speaker may not be the current MP	App warns when an electorate's latest speaker spoke before 2025

Table 3. Data quality issues, evidence and handling.

4. NLP methods and techniques, with justification

Figure 1 shows the system. All methods were chosen from, or are direct extensions of, the techniques taught in the subject, and each choice was constrained by the same practical facts: a CPU-only laptop, no paid API access, and a requirement that every output shown to a citizen can be traced to the official record.

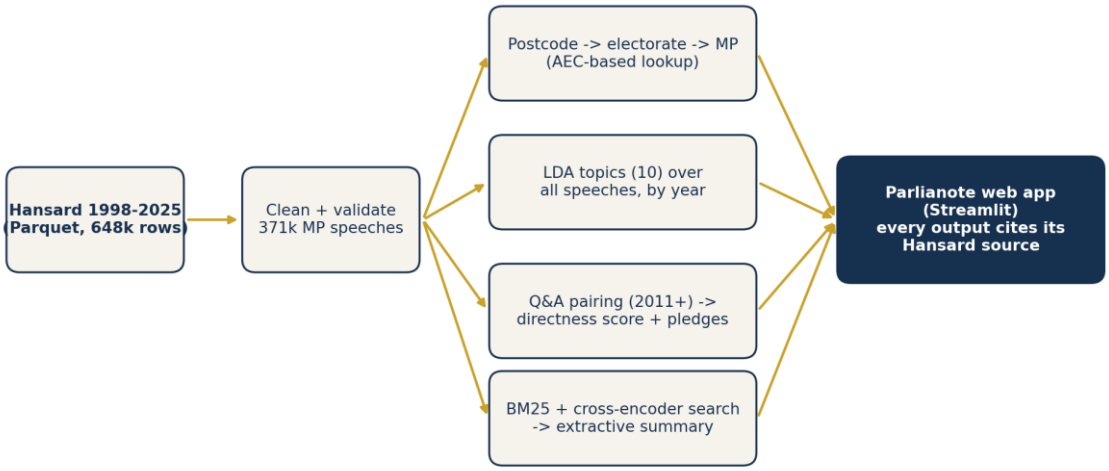


Figure 1. System overview. The Hansard corpus is cleaned once; four independent components feed one web application.

Technique	Role in Parlianote	Why this choice	Alternative considered
Tokenisation, stop-word removal (Sessions 1-2)	Prepare text for LDA and TF-IDF	Standard, transparent; we added 45 parliamentary filler words ("member", "minister", "house") that otherwise dominate every topic	Lemmatisation (little gain for topic words)
LDA topic model (Session 2)	What an MP talks about, and when	Unsupervised, interpretable, cheap enough to run on 247,011 speeches on CPU	BERTopic (heavier); fixed-issue zero-shot (tested, covers only 42% of speeches)
Sentence embeddings, all-MiniLM-L6-v2 (Sessions 3, 8)	Semantic similarity for issue tagging, answer relevance and sentence selection	Small and fast on CPU, strong general-purpose quality (Reimers and Gurevych, 2019; Wang et al., 2020)	Larger embedding models (too slow for 1.7M passages)
BM25 + cross-encoder re-ranking (Session 8)	Find the passages that answer "what did my MP say about X?"	BM25 is exact and instant on one MP's text; the cross-encoder reads query and passage together and corrects its ranking on only 60 candidates	Dense search over every passage (about 9 hours of CPU embedding for the corpus)
Extractive summarisation (Session 8, RAG)	Short answer made only of verbatim sentences	Every sentence can be checked against Hansard; no hallucination; no API cost	Generative summaries from an LLM (future work)
Supervised classifiers (Sessions 3-4)	Answer directness; pledge type	Small labelled data suits linear models with explicit features; coefficients can be inspected	Fine-tuned transformer (not enough labels); LLM judge (no API)

Table 4. Techniques, justification and alternatives.

4.1 Pre-processing and sampling

Text is lower-cased, tokens are alphabetic strings of three or more letters, and English stop words plus 45 parliamentary filler words are removed. For model development we drew a stratified sample of 19,600 speeches of 80 to 3,000 words (700 per year, 1998 to 2025, fixed seed) so that no year dominates. The final topic model is then applied to all 247,011 speeches of 80 words or more.

4.2 Topic modelling and issue classification

LDA (Blei et al., 2003) with 10 topics, 8 passes and a fixed seed was fitted on the 19,600-speech sample with gensim (Rehurek and Sojka, 2010). Topic quality was measured with the c_v coherence score (Roder et al., 2015). We named each topic from its ten highest-weight words (Table 5). Only k = 10 was trained; a sweep over k is future work.

Zero-shot issue classification (phase 1). We also tested whether speeches could be assigned to twelve fixed issues (health, economy, immigration and so on) without labelled data, by embedding a one-line description of each issue and each speech and choosing the nearest issue by cosine similarity. The naive version was biased: one category (Indigenous Affairs) absorbed 5,965 of 19,600 speeches (30%), including a speech about copyright law,

because raw cosine scores are not comparable across categories. We fixed this by z-scoring each category's similarity across the corpus and requiring both z >= 1.0 and raw similarity >= 0.28; anything below is labelled Other/Uncertain. Because this labels only 41.8% of speeches, the application uses LDA, which assigns a topic to every substantive speech, and keeps the issue classifier as an analysis tool.

4.3 Retrieval and extractive summarisation

Each MP's speeches are split into overlapping 110-word passages (stride 90). For a query we (1) score every passage with BM25 (Robertson and Zaragoza, 2009) and keep the top 60, (2) re-rank those 60 with the ms-marco MiniLM cross-encoder (Nogueira and Cho, 2019, describe the approach), and (3) keep the top eight. The summary is the three sentences from those passages that are closest to the query in embedding space, shown in date order with a Hansard citation (date, page, speech number) under each. A year slider restricts the search, and the best early and late passages are shown side by side so a user can see how wording changed. We evaluate three retrieval variants in Section 5.4.

We deliberately do not generate text. A generative summary cannot be checked sentence by sentence, would need an API key or a GPU, and was the main source of unverifiable claims in our phase-1 pilot, where one fully hand-written summary had to be checked claim by claim. Extractive output keeps the "every claim links to the record" promise true by construction.

4.4 Estimating how directly an answer responds

Pairing. For each Question Time question we take the next answer row on the same sitting day before the next question, giving 16,256 pairs from 2011 to 2025. **Labels.** One annotator labelled 120 randomly sampled pairs as Direct (addresses the specific thing asked), Partial (on topic but not the specific ask) or Evasive (deflects, attacks, refers to earlier answers or ignores the question). **Features.** Lexical cues (attack phrases such as "those opposite", references to earlier answers, opening with thanks, yes/no openings, digits, lengths, question marks, TF-IDF overlap), bi-encoder cosine similarity between the question and the answer's opening, best and mean 100-word chunk, and cross-encoder relevance scores for the same chunks. **Models.** A majority-class baseline; TF-IDF with a linear SVM on the first 120 answer words (classical); logistic regression on lexical cues (classical), on embedding similarity, on cross-encoder scores, and on all features (semantic and combined). **Protocol.** Repeated stratified 5-fold cross-validation (10 repeats, class-weighted), reporting accuracy and macro-F1. Because friendly (same-party) questions are almost always answered directly, we report results on all pairs and on opposition questions only.

We did not use a large language model as a judge or an NLI model: neither was available without an API key or GPU, and the cross-encoder gave us a joint reading of question and answer on CPU. This is the main way a stronger system could improve.

4.5 Pledge detection

A sentence is a **pledge** if it matches a first-person or government future pattern ("we will", "I will", "are committed to", "guarantee") and a **specific pledge** if it also contains a number, date, dollar amount or frequency that can be checked later; hedged forms ("will consider", "look at") are general. The baseline is simply "the sentence contains will". We compared the rules with logistic regression on sentence embeddings using 90 labelled sentences.

4.6 Web application design

The application is a Streamlit app (Figures 6 and 7) reading pre-computed Parquet files, with each MP's speeches loaded on demand through column filters so memory use stays small. The flow is: enter a postcode (or pick an MP by name) -> MP card -> five views (Overview, Ask the record, Question Time, Pledges, About and limits). Design decisions follow from the evaluation:

- **Every quoted sentence carries its citation** (date, page, speech number) and a link to the official Hansard site, so a user can check it themselves.

- **Uncertainty is shown, not hidden.** Directness is compared only on opposition questions, requires at least 10 exchanges, and is displayed with a Wilson 95% confidence interval (Wilson, 1927) next to the all-minister baseline.
- **Neutral wording.** The interface says the scores describe language patterns, not intent, and never ranks MPs.
- **Edge cases are handled in the interface:** postcodes that span several electorates, seats whose latest speaker is old, MPs with too few exchanges, and a limits page that restates the data cut-off and annotation limits.
- **Shareable links.** Query parameters (postcode, view, search text) preselect a screen, which also let us test each view reproducibly.

5. Findings

All accuracy figures below are estimates from small, single-annotator samples and should be read with the uncertainty stated; Section 5.7 summarises what the evidence does and does not support.

5.1 What MPs talk about, and how it has changed

The LDA model reached a c_v coherence of **0.459**. Table 5 lists the ten topics, named from their highest-weight words, and the share of the 247,011 speeches of 80 words or more that each accounts for (the other 123,850 speeches are shorter and receive no topic). The largest single topic, at 24.6%, is partisan debate ("labor, party, prime, opposition"), so a quarter of substantive speech is about the contest between parties rather than a policy area.

Topic (our name)	Highest-weight words	Share
Regional & local infrastructure	regional	NaN%
Health	aged care & energy	NaN%
Community & family life	community	NaN%
Legislation & amendments	act	NaN%
Defence & veterans	defence	NaN%
Economy	tax & budget	NaN%
Partisan debate	labor	NaN%
Industry	trade & agriculture	NaN%
Rights	safety & security	NaN%
Education & training	education	NaN%

Table 5. LDA topics, top words and share of topic-labelled speeches (all MPs, 1998-2025).

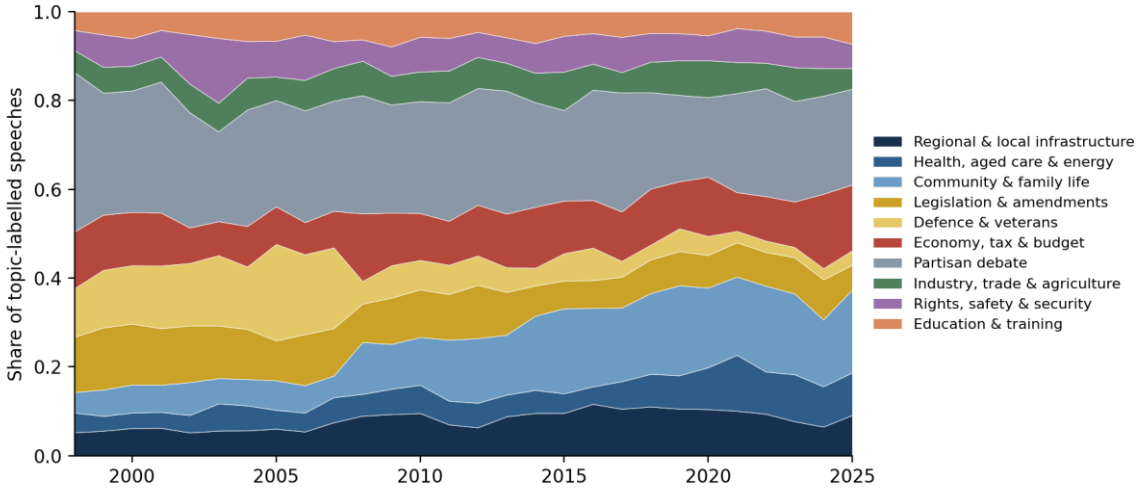


Figure 2. Share of topic-labelled speeches by year, all MPs. Partisan debate fell from 36% of speeches in 1998 to 18% in 2020 (22% in 2025); community and family life rose from 5% to 19%; defence and veterans fell from 11% to 3%; legislation and amendments halved from 12% to 6%.

These shifts are descriptive. Some may reflect changes in parliamentary practice (for example more short constituency statements) rather than changes in what MPs care about, and we did not test alternative numbers of topics.

5.2 Fixed-issue classification

With raw cosine similarity, one category absorbed 30% of all speeches; after z-score calibration and thresholds the twelve issues were spread more evenly (Figure 3) and 8,186 of the 19,600 sampled speeches (41.8%) received a confident label. Grading 40 random confident labels by hand, 31 were correct (**77.5%**, 95% Wilson interval 62.5% to 87.7%). Five of the nine errors were the same category, Indigenous Affairs, being applied to generic community language (a sports club, a jobs summit, community radio, a bushfire response), a systematic bias rather than random noise. Because 58% of speeches fit none of the twelve issues, we did not use this classifier in the application.

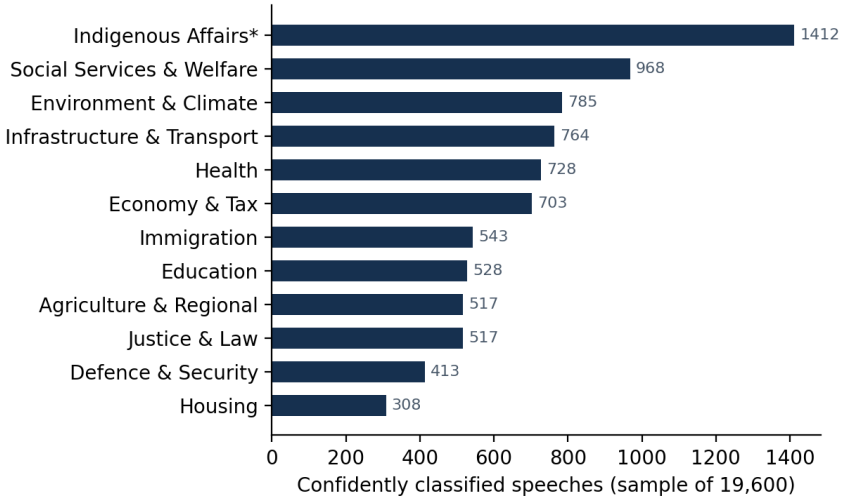


Figure 3. Confidently classified speeches by issue (phase-1 sample of 19,600). *Indigenous Affairs includes the systematic false positives described above.

5.3 How directly do ministers answer?

The strongest pattern in our labels is not about the answer at all: of 56 pairs where the question came from the same bloc as the minister, 45 were labelled Direct, 9 Partial and only 2 Evasive, whereas of 64 pairs from another bloc only 7 were Direct, 30 Partial and 27 Evasive (Figure 4). Friendly "Dorothy Dixer" questions are almost always

answered directly. Any model trained on all pairs therefore partly learns who is asking, so we also evaluate on other-bloc questions only.

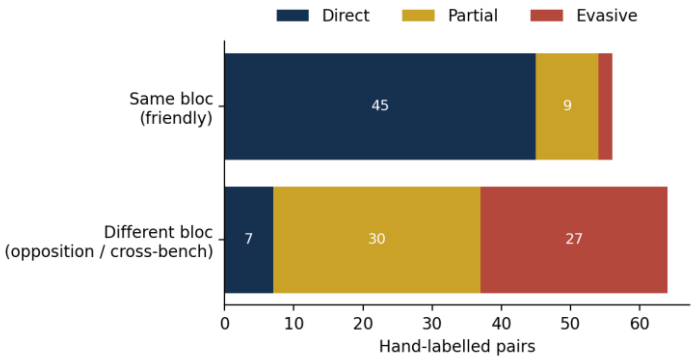


Figure 4. Hand labels by question type (120 pairs). Same bloc = questioner and minister are both Labor or both Coalition (Liberal and National counted together).

Model	All pairs (n=120) 3-class acc / macro-F1	All pairs evasive vs not acc	Other-bloc (n=64) 3-class acc / macro-F1	Other-bloc evasive vs not acc
Majority class	0.43 / 0.20	0.76	0.47 / 0.21	0.58
TF-IDF + SVM	0.56 / 0.53	0.80	0.61 / 0.42	0.68
Lexical cues + LR	0.63 / 0.61	0.77	0.61 / 0.51	0.68
Embedding similarity + LR	0.44 / 0.43	0.67	0.46 / 0.41	0.59
Cross-encoder + LR	0.59 / 0.58	0.70	0.48 / 0.39	0.69
All features + LR	0.64 / 0.62	0.77	0.61 / 0.48	0.68

Table 6. Repeated 5-fold cross-validation (10 repeats), mean over folds. Fold standard deviations are 0.06 to 0.14, so most differences between non-baseline models are not reliable.

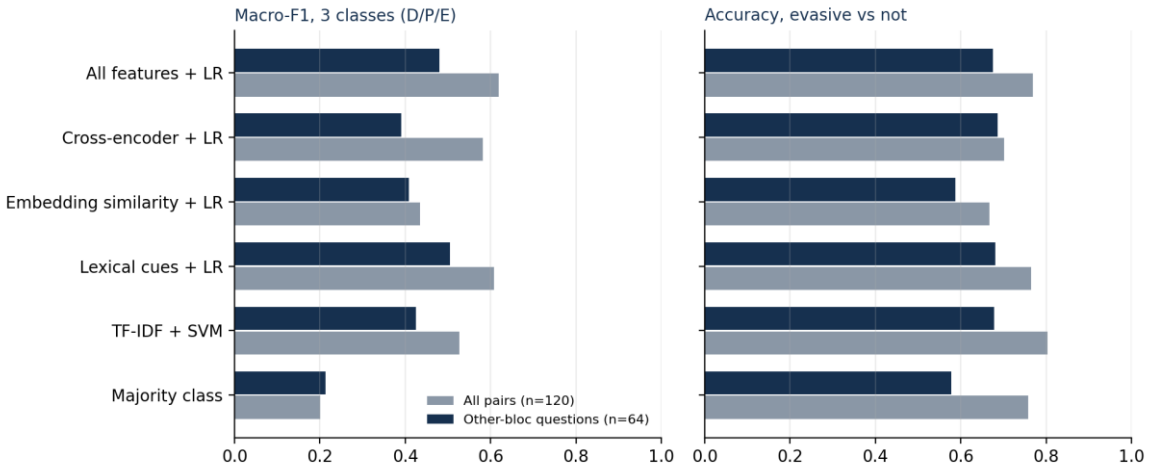


Figure 5. Classical versus semantic approaches. Left: macro-F1 for the three-class task; right: accuracy for evasive-versus-not. Grey bars use all pairs, navy bars use other-bloc questions only.

Reading the results. (1) On all pairs, lexical cues with logistic regression reach 0.63 accuracy against a 0.43 majority baseline. (2) On other-bloc questions, which is the case that matters, every classical or combined model reaches about 0.61 three-class accuracy against 0.47, and about 0.68 against 0.58 for evasive versus not: roughly ten points above baseline with a standard deviation near ten points. (3) **Bi-encoder similarity between question and answer is no better than the baseline** (0.59): an evasive answer usually still talks about the same subject, so topical similarity cannot detect evasion. (4) The cross-encoder, which reads question and answer together, had the numerically highest two-class accuracy on other-bloc questions (0.69), but the gap to the lexical and TF-IDF models

is far smaller than the fold-to-fold variation, and it was weaker for three classes. We deployed the all-features model because it is the most complete, not because it is demonstrably better.

Applying the model to all 16,020 pairs (excluding questions put to the Speaker) it labels 95% of friendly questions Direct, and 53% of other-bloc answers Evasive, 43% Partial and 4% Direct. The predicted evasive share is higher than the 42% in our labelled sample (27 of 64), so the model over-predicts evasion and absolute rates should not be quoted; comparisons are only meaningful relative to the all-minister baseline of 53%. Of the 31 ministers with at least 40 other-bloc answers, 8 have an interval entirely above the baseline and 11 entirely below it. We do not publish a ranking: the model has not been validated against an independent measure, and the pattern is confounded with era and role (for example the opposition-facing style of a prime minister versus a portfolio minister).

5.4 Retrieval: what did my MP say about X?

We compared three retrievers on 15 queries across five MPs (Steggall, Bandt, Plibersek, Wilkie, Tink): **BM25 alone**, **BM25 followed by cross-encoder re-ranking** (the application's method) and **dense embedding search** with MiniLM over every passage. Ten queries used wording an MP would plausibly use ("paid parental leave", "poker machines and gambling harm"); five were paraphrases where the MP's own words differ from the query ("support for mums and dads with newborn babies", "cutting the rubbish that ends up in landfill"). For each query we pooled the top three passages of every method, removed duplicates, hid which method found each, and judged every pooled passage relevant or not (relevant means the MP, in their own words, substantively discusses the topic). Table 7 gives precision at 3.

Method	All 15 queries (45 slots)	10 keyword queries (30 slots)	5 paraphrase queries (15 slots)
BM25 only	35 of 45 (0.78)	30 of 30 (1.00)	5 of 15 (0.33)
BM25 + cross-encoder (ours)	43 of 45 (0.96)	30 of 30 (1.00)	13 of 15 (0.87)
Dense MiniLM search	38 of 45 (0.84)	26 of 30 (0.87)	12 of 15 (0.80)

Table 7. Relevant passages in the top three, and precision@3 in brackets. One annotator, blind to method.

Findings. BM25 is perfect when the query uses the MP's words (30 of 30) but fails on paraphrases (5 of 15) because there is little word overlap to match. Dense search is the opposite: it copes with paraphrase (12 of 15) but drifts to neighbouring topics on keyword queries (26 of 30), for example returning unpaid parental leave for "paid parental leave" and online gambling for "poker machines". The hybrid combines exact matching with a model that reads query and passage together and was relevant in 43 of 45 slots (0.96). Its two misses were paraphrase queries where BM25 supplied few useful candidates; because re-ranking only reorders BM25's top 60, it cannot recover passages that share no words with the query, which is the main limit of the design. With 15 queries and one annotator, differences of one or two passages are within noise, but the pattern is consistent across queries.

In our phase-1 pilot, a dense FAISS index over one MP's speeches returned relevant passages for all 24 of the top-3 slots across 8 queries; that pilot used one MP and mostly keyword-style queries, so it is a feasibility check rather than a comparison.

5.5 Pledge detection

The 90 labelled sentences were drawn half from sentences flagged by the first version of our rules and half from unflagged ones (3.8% of 33,761 sentences were flagged), so every estimate below re-weights each sentence by its stratum. The naive baseline, "the sentence contains *will*", flags 44 of the 90 but has a population precision of only **0.51**: about half of all *will* sentences are predictions about opponents or description rather than pledges. After the final rule changes (which exclude other people's quoted words and bare nouns such as "the superannuation guarantee") the rules flag 43 sentences with a population precision of **0.79** and a recall of **about 0.40**; the recall estimate is imprecise because it rests on 2 commitments found among 45 unflagged sentences. Of the 43 flagged sentences, 24 were only vague or hedged intentions ("we will continue to deliver"), which is why a specificity cue is useful: requiring a number, date, dollar amount or frequency left 7 sentences, 6 of which were specific

commitments (precision 0.86), recovering 6 of the 12 specific commitments in the sample (raw recall 0.50). Population recall for specific pledges cannot be estimated reliably because it depends on a single unflagged specific commitment. Logistic regression on sentence embeddings reached 0.73 three-class accuracy and 0.62 macro-F1 in cross-validation, against 0.60 and 0.44 for the rules, but the sample is small and we kept the rules in the application because their output can be explained to a user.

5.6 The application

We exercised all five views in a browser on a CPU-only Windows laptop. **Responsiveness:** the language models load once per server start (about 14 s); the first search for an MP builds a BM25 index in 0.7 s (Steggall, 611 speeches, 5,547 passages) to 2.4 s (Albanese, 8,377 speeches, 28,846 passages); each search then takes 1.2 to 1.5 s including re-ranking and sentence selection. **Postcode lookup:** Table 8 shows nine postcodes that resolved to the expected seat and MP, including two seats that changed hands at the May 2025 election (Melbourne and Brisbane). Brisbane returned the previous MP before we fixed the member-flag problem. The postcode dataset is approximate: it places postcode 2060 (North Sydney) in Warringah, which we have not been able to confirm and which is why the app asks users to confirm their electorate with the AEC when a postcode spans seats.

Postcode	Electorate	MP returned	Note
2088	Warringah	Zali Steggall (IND)	
2000	Sydney	Tanya Plibersek (ALP)	
2040	Grayndler	Anthony Albanese (ALP)	
3000	Melbourne	Sarah Witty (ALP)	Changed hands, 2025
3141	Macnamara	Josh Burns (ALP)	
4000	Brisbane	Madonna Jarrett (ALP)	Changed hands, 2025
6000	Perth	Patrick Gorman (ALP)	
2600	Canberra	Alicia Payne (ALP)	
7000	Clark	Andrew Wilkie (IND)	Seat formerly Denison

Table 8. Postcode-to-MP spot checks (nine capital-city postcodes). This is a spot check, not a full validation against the AEC.

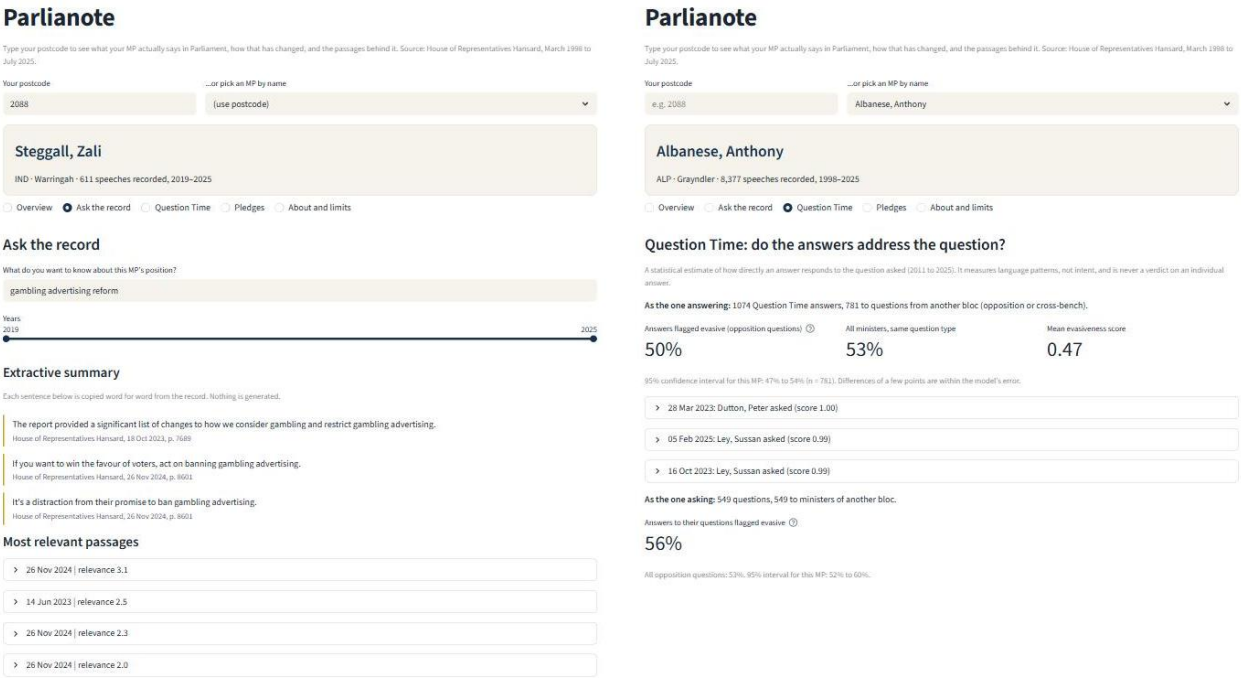


Figure 6. The application. Left: Ask the record returns a verbatim extractive summary, each sentence with its Hansard date and page, then the most relevant passages. Right: Question Time shows the share of answers estimated evasive on other-bloc questions beside the all-minister baseline, with a 95% interval and the highest-scoring exchanges for inspection.

We did not run a user study. The interface choices that matter for usability and trust are therefore justified by design reasoning (Section 4.6): citations beside every quotation, plain-language labels, uncertainty shown with every estimate, and an explicit limits page. A task-based test with members of the public is the most important missing evaluation.

5.7 What the evidence supports

Component	Supported by evidence	Not supported / caution
Postcode to MP	Correct on all spot-checked postcodes after the member-flag fix; 86% of Hansard electorates match current postcode data	Seats that changed hands after the 3 May 2025 election may show the previous MP; postcodes can span electorates
Topics over time	Coherent, interpretable topics (c_v 0.459); clear long-run shifts	No sweep over k; topic labels are ours; 33% of speeches too short for a topic
Search and extractive summary	Every sentence is verbatim and cited; hybrid retrieval relevant in 43 of 45 blind-judged slots vs 38 (dense) and 35 (BM25)	Fifteen queries, one annotator; cannot recover passages BM25 misses; no user study
Directness estimate	Roughly 10 points above baseline on other-bloc questions	Single annotator, n=64 for the hard case, over-predicts evasion, not validated externally; not suitable for ranking people
Pledges	Rules beat the "contains will" baseline (precision 0.79 vs 0.51); specificity cue 6 of 7 correct	Recall about 0.40; 90 labelled sentences; embeddings scored higher in cross-validation

Table 8. Summary of evidence by component.

6. Outcomes and value added

What was delivered. (1) A working web application in which a postcode leads to the MP, a topic profile over time, cited search with an extractive summary, a Question Time directness estimate and a list of pledges, covering 490 MPs and 370,861 speeches. (2) A reproducible pipeline (src/) from the raw Parquet file to the app's data, with every evaluation script and result table in results/. (3) Two small labelled datasets (120 question-answer pairs, 90 sentences) and a ready-to-use sheet for independent annotation. (4) A data-quality audit that corrected an error in our own earlier analysis.

Value for users. Vote trackers show how an MP voted and keyword tools show where a word appears. Parlianote shows what the MP said, in their own words and in context, and makes it a one-click step to check the source. A citizen can go from a postcode to a cited answer about their MP's position on housing or climate in under a minute; an advocacy group or journalist can compare how that position was worded in different years. The deliberate choice of extractive summaries means that nothing shown is invented, which matters more for a tool about political speech than fluent prose would.

Value as a piece of applied NLP. The project produced a clear negative result that a less careful evaluation would have hidden: question-answer *similarity* does not measure evasiveness, and a classifier that looks accurate on all pairs mostly learns who asked the question. Reporting this, with the evidence, is a more honest contribution than a headline accuracy.

What we do not claim. We have not run a user study, so claims about ease of use rest on design principles and our own testing. We have not validated the directness scores against independent annotators or an external measure, and the application is not deployed publicly.

7. Challenges faced and solutions

Challenge	What happened	Solution
Biased zero-shot issue classifier	Raw cosine similarity put 30% of speeches into one category, including unrelated ones	Z-score calibration per category plus a confidence threshold with an Other/Uncertain class; 41.8% coverage, 77.5% precision

Challenge	What happened	Solution
Silent loss of recent MPs	The member flag is blank for MPs elected from 2022, so 8,658 speeches (69 MPs) were dropped; found when the Brisbane postcode returned the wrong MP. The same MPs also lack a uniqueID	Filter on member != 0; use the display name as a fallback MP key; re-ran topic inference on all 370,861 speeches
Unreliable answer flags before 2011	Few answers were flagged before 2011, so questions had no partner	Restrict Question Time analysis to 2011-2025 and say so in the interface
Evaluation shortcut	Friendly questions are nearly always answered directly, inflating accuracy; party matching also mislabelled Liberal-National pairs as hostile and distorted minister rates	Define friendly questions by bloc, report other-bloc results separately, exclude Speaker answers, recompute all results
No labelled ground truth	No dataset marks answers as evasive	Built a labelling scheme and labelled 120 pairs; report wide uncertainty; prepared an independent labelling sheet
Overclaiming risk	Our poster described inter-annotator agreement and a bias audit that had not been run	This report states exactly what was done; agreement and bias audit are listed as next steps
Compute and cost	CPU-only machine and no API key: a dense index of about 1.7M passages needs roughly 9 hours; generative summaries were not possible	BM25 first stage plus cross-encoder on 60 candidates; extractive summaries
Tooling on Windows	Gensim multicore LDA crashed under Windows process spawning; the Streamlit file watcher imported every transformers module	Single-process LDA training with a separate parallel inference module; file watcher disabled

Table 9. Challenges and solutions.

8. Recommendations and next steps

1. **Independent annotation (highest priority).** Have at least two people who did not build the model label 300 or more pairs, report Cohen's kappa, and re-estimate every directness result. The sheet and script already exist (team_labelling/, src/compute_kappa.py).
2. **Stronger directness models.** Fine-tune an NLI or cross-encoder model on the larger labelled set, or evaluate an LLM judge with its known biases controlled (position, length, self-preference), and compare against the classical baselines using the same protocol.
3. **Topic modelling.** Sweep the number of topics, compare with BERTopic, and check whether the long-run shifts survive.
4. **Generative summaries with guardrails.** Add an optional LLM summary that must quote retrieved sentences, and test faithfulness with claim-level checking and ROUGE/BERTScore against the extractive summary.
5. **Coverage.** Include the Senate, committee evidence and Question Time before 2011 (re-parsing the XML), and update the corpus automatically as new Hansard is published.
6. **User testing and deployment.** Run a task-based study with members of the public (for example the System Usability Scale), then deploy with monthly refreshes and an explicit correction channel.
7. **Responsible use.** Keep the language-patterns-not-intent framing, never rank individuals on the directness estimate, and give MPs a way to flag errors.

9. References

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research, 3*, 993-1022.

Johnson, J., Douze, M., & Jegou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data, 7*(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572

Katz, L., & Alexander, R. (2025). *A new, comprehensive database of all proceedings of the Australian Parliamentary Debates (1998-2025)* [Data set]. Zenodo. https://doi.org/10.5281/zenodo.17351233

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459-9474.

Nogueira, R., & Cho, K. (2019). *Passage re-ranking with BERT* (arXiv:1901.04085). arXiv. https://arxiv.org/abs/1901.04085

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Proctor, M. (n.d.). *Australian postcodes* [Data set]. https://www.matthewproctor.com/australian_postcodes

Rehurek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45-50.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982-3992. https://doi.org/10.18653/v1/D19-1410

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333-389. https://doi.org/10.1561/1500000019

Roder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining*, 399-408. https://doi.org/10.1145/2684822.2685324

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems, 33*, 5776-5788.

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209-212. https://doi.org/10.1080/01621459.1927.10502953

Appendix A. Reproducing the results

All code is in the project folder. Run from the project root (Python 3.13; packages: pandas, pyarrow, gensim, nltk, sentence-transformers, scikit-learn, rank-bm25, streamlit, altair, matplotlib).

Step	Command	Output
1. Phase-1 analysis	Run notebooks/parlianote_analysis.ipynb	Cleaning, postcode join, LDA, zero-shot classifier, retrieval pilot
2. Question-answer pairs and label samples	python -m src.build_pairs, then python -m src.make_label_samples	data/qa_pairs.parquet (16,256 pairs); samples to label
3. Labelled features	python -m src.build_label_features	data/evasiveness_label_features.parquet
4. Evaluation tables and figures	python -m src.make_figures	results/model_comparison.csv, figures
5. Score all pairs	python -m src.score_pairs, then python -m src.rescore	data/qa_pairs_scores.parquet
6. Topic model	python -m src.train_lda	models/lda_model.gensim (c_v 0.4592)
7. App data	python -m src.prepare_app_data, python -m src.merge_scores	app/data/*.parquet
8. Retrieval evaluation	python -m src.run_retrieval_eval pool, then score	results/retrieval_comparison.csv
9. Pledge evaluation	python -m src.run_commitment_eval	reweighted precision and recall
10. Run the app	streamlit run app/app.py	http://localhost:8501/?postcode=2088

Table A1. Reproduction steps. Random seeds are fixed (LDA 42; sampling 42, 11, 7; cross-validation 1).

Appendix B. Annotation guidelines and examples

Direct (D): the answer addresses the specific thing asked, even if it adds spin. **Partial (P):** the answer stays on the topic but not on the specific ask, or answers only part. **Evasive (E):** the answer deflects, attacks the opposition, refers to earlier answers, or ignores the question. Labels were assigned from the question and the first 110 words of the answer, so late-answer content is not seen.

Label	Question (abridged)	Answer (abridged)
D	Is the Prime Minister aware ... that the Prime Minister's discredited forced divestment policy sets a dangerous precedent ...? Does the Prime Minister agree? (Rowland, 2018)	No, I don't. I don't support that. What the ACCC retail electricity pricing inquiry final report found ... (Morrison)
P	Does the Prime Minister agree with his Treasurer? Will the Prime Minister join with Labor to ensure that teachers can't be sacked because of who they are or who they love? (Shorten, 2018)	I'm happy to table the letter that I sent to the Leader of the Opposition today ... it's very important that we act in this parliament over the next fortnight ... (Morrison)
E	Senator Canavan resigned from cabinet 43 days ago. Why is the Deputy Prime Minister still sitting on your front- (Shorten, 2017)	The honourable member's question has a distinctly familiar ring to it, and I refer him to my earlier answers. (Turnbull)

Table B1. Worked examples (House of Representatives Hansard, quoted from the public record).

Inter-annotator agreement has not been measured. All 120 pair labels and 90 sentence labels come from one annotator (the AI assistant, Section 2). A 45-item sheet for independent annotation (team_labelling/labelling_sheet.xlsx) and the agreement script (src/compute_kappa.py) are ready; until at least two humans complete it, the accuracy figures in Section 5 should be treated as preliminary.

Appendix C. Key code

Friendly-question definition (src/blocs.py)

```
BLOC = {"ALP": "Labor", "LIB": "Coalition", "NP": "Coalition", "NAT": "Coalition", "LNP": "Coalition", "CLP":
"Coalition"}
def bloc(party): return BLOC.get(str(party), "Other")
# friendly question  <=>  bloc(questioner party) == bloc(minister party)
```

Hybrid retrieval and extractive summary (src/retrieval.py)

```
def bm25_search(ps, bm, query, k=60, year_range=None):
```

```
    scores = bm.get_scores(tokens(query))
    top = [i for i in np.argsort(-scores)[:k] if scores[i] > 0]
    return ps.iloc[top].assign(bm25=scores[top])


def rerank(ce, query, cand, k_final=8):
    cand["rerank"] = ce.predict([(query, t) for t in cand["text"]], batch_size=32)
    return cand.sort_values("rerank", ascending=False).head(k_final)


def extractive_summary(bi, query, hits, n=3):   # n verbatim sentences closest to the query
    sents = [(i, s) for i, r in hits.iterrows() for s in split_sentences(r["text"])]
    sim = bi.encode([s for _, s in sents], normalize_embeddings=True) @ bi.encode([query],
normalize_embeddings=True).T
    ...  # pick top n, drop near-duplicates, return in date order with citations
```

Directness model and evaluation protocol (src/make_figures.py, src/rescore.py)

```
lr = lambda: make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000, class_weight="balanced",
C=0.5))
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=1)
for tr, te in cv.split(Xs, yt):
    m = clone(clf).fit(Xs.iloc[tr][cols], yt[tr]); p = m.predict(Xs.iloc[te][cols])
    acc.append(accuracy_score(yt[te], p)); mf.append(f1_score(yt[te], p, average="macro"))
# features: LEX (lexical cues), SEM (bi-encoder cosine), CE (cross-encoder relevance) + same_bloc
```

Pledge detection (src/commitment.py)

```
FIRM = re.compile(r"\b(we|i|the government|this government|our government|the coalition|labor|the
minister)\s+"
                  r"(will|shall|are going to|have committed to|are committed to|commit(s)?
to|guarantee|promise|pledge)\b", re.I)
SPECIFIC = re.compile(r"(\d|\$|\bby (the end of )?(next|this|20\d\d)|\bevery (week|year|month)|\bper
cent\b)", re.I)
def commitment_type(s):
    if is_commitment(s) == "N": return "none"
    return "specific pledge" if SPECIFIC.search(s) else "general pledge"
```